

GENERADOR DE NÚMEROS PSEUDOALEATORIOS EN JAVA

Método Congruencial Cuadrático

Juan David Roa Ahumada

Juan David Sabogal Arenas

Jose Luis Leudo Mosquera

Docente:

Simar Enrique Herrera Jiménez

UNIVERSIDAD FRANCISCO JOSÉ DE CALDAS

Ciencias de la Computación II

Bogotá D.C., Colombia

2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. ¿Cómo se logró obtener la función?	2
4. ¿Cómo funciona la función?	3
4.1. Fórmula matemática	3
4.2. Estructura del código	4
4.3. Explicación paso a paso	4
5. Pruebas	5
6. Conclusiones	8

1. Introducción

Este documento presenta la documentación técnica de un programa desarrollado en el lenguaje Java, cuyo propósito es generar una secuencia de números pseudoaleatorios mediante la implementación de un algoritmo generador de tipo congruencial. El programa está compuesto por dos clases principales: `Main.java`, que actúa como punto de entrada de la aplicación, y `NumerosPseudoaleatorios.java`, que contiene la lógica del algoritmo generador.

Los números pseudoaleatorios son secuencias de valores que, aunque generados a partir de un proceso completamente determinístico, presentan propiedades estadísticas similares a las de una secuencia verdaderamente aleatoria.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar un programa en Java capaz de generar una secuencia de números pseudoaleatorios a partir de una fórmula matemática recursiva, con el fin de comprender el funcionamiento interno de los generadores de números pseudoaleatorios (PRNG, por sus siglas en inglés).

2.2. Objetivos Específicos

- Implementar una clase en Java que encapsule los parámetros necesarios para un generador congruencial (semilla, multiplicador, incremento y módulo).
- Aplicar una fórmula recursiva que permita transformar un valor semilla en una nueva secuencia numérica en cada iteración.
- Generar y mostrar por consola un conjunto de 100 números pseudoaleatorios como resultado de la ejecución del algoritmo.
- Analizar el comportamiento y las limitaciones del método implementado en cuanto a periodicidad y distribución de los valores obtenidos.

3. ¿Cómo se logró obtener la función?

Para el desarrollo de esta función generadora se partió del método congruencial de generación de números pseudoaleatorios visto en clase, el cual genera una secuencia numérica a partir de una relación de recurrencia; es decir, cada nuevo valor depende del valor generado inmediatamente antes.

El proceso seguido para construir la función fue el siguiente:

1. **Definición de los parámetros del algoritmo:** se establecieron cuatro variables de tipo `long` que actúan como constantes de configuración del generador:
 - **semilla:** valor inicial (o valor actual en cada iteración) sobre el cual se calcula el siguiente número. Valor inicial: 520.
 - **multiplicador:** constante que pondera linealmente a la semilla. Valor: 21.
 - **incremento:** constante aditiva de la fórmula. Valor: 7.

-
- **modulo:** valor sobre el cual se aplica la operación módulo para acotar el rango de salida. Valor: 16383.
2. **Selección de la fórmula recursiva:** a diferencia del método congruencial lineal clásico ($X_{n+1} = (aX_n + c) \bmod m$), se optó por una variante **cuadrática**, incorporando el término X_n^2 dentro de la operación, lo que introduce un componente no lineal al cálculo y modifica el comportamiento estadístico de la secuencia resultante.
 3. **Implementación en Java:** se creó la clase `NumerosPseudoaleatorios`, cuyo constructor inicializa los cuatro parámetros mencionados, y se definió el método `funcion()`, encargado de ejecutar la fórmula de manera iterativa dentro de un ciclo `for`, actualizando el valor de la semilla en cada paso y mostrando el resultado por consola.
 4. **Prueba de la implementación:** desde la clase `Main`, se instanció un objeto de tipo `NumerosPseudoaleatorios` y se invocó el método `funcion()` para verificar que la secuencia de 100 números se generara correctamente en tiempo de ejecución.

4. ¿Cómo funciona la función?

4.1. Fórmula matemática

El núcleo del generador se basa en la siguiente expresión matemática, aplicada de forma recursiva:

$$X_{n+1} = (X_n^2 + a \cdot X_n + c) \bmod m \quad (1)$$

donde:

- X_n es la semilla actual (el número pseudoaleatorio previamente generado).
- a es el multiplicador (`multiplicador = 21`).
- c es el incremento (`incremento = 7`).
- m es el módulo (`modulo = 16384`), que determina el rango de los valores generados: $[0, m - 1]$.
- X_{n+1} es el nuevo número pseudoaleatorio, el cual se convierte en la semilla para la siguiente iteración.

4.2. Estructura del código

El código fuente completo de la clase generadora es el siguiente:

```
1 public class NumerosPseudoaleatorios {
2     public long semilla;
3     public long multiplicador;
4     public long modulo;
5     public long incremento;
6
7     public NumerosPseudoaleatorios(){
8         semilla = 520;
9         multiplicador = 21;
10        modulo = 16384;
11        incremento = 7;
12    }
13
14    public void funcion() {
15        for (int i = 0; i < 100; i++) {
16            semilla = (semilla*semilla+multiplicador*semilla+incremento) %
modulo;
17            System.out.println(semilla);
18        }
19    }
20 }
```

Listing 1: Clase NumerosPseudoaleatorios.java

Y la clase principal que ejecuta el programa:

```
1 public class Main {
2     public static void main(String[] args) {
3         NumerosPseudoaleatorios p1 = new NumerosPseudoaleatorios();
4         p1.funcion();
5     }
6 }
```

Listing 2: Clase Main.java

4.3. Explicación paso a paso

1. Al invocar el método `funcion()`, se ejecuta un ciclo `for` que itera exactamente 20 veces.
2. En cada iteración, la variable `semilla` se recalcula aplicando la fórmula: se eleva al cuadrado su valor actual, se le suma el producto del multiplicador por la semilla, se le suma el incremento, y finalmente se aplica el operador módulo respecto a `modulo`.
3. El resultado de esa operación reemplaza el valor anterior de `semilla`, la cual funciona simultáneamente como entrada y salida del algoritmo (es decir, el nuevo número depende únicamente del número anterior).
4. El valor calculado se imprime inmediatamente por consola mediante `System.out.println()`.
5. Este proceso se repite hasta completar las 100 iteraciones, produciendo una secuencia de 20 números pseudoaleatorios comprendidos en el rango `[0, 16383]`.

5. Pruebas

A continuación se presentan los resultados obtenidos al ejecutar el generador: la secuencia completa de los 100 números generados, y las pruebas de uniformidad realizadas sobre los primeros 20 números y sobre el total de 100 números.

Cuadro 1: Números generados y repeticiones (100 iteraciones)

Iteraciones	Números generados	Repeticiones
1	2799	1
2	12483	1
3	13455	1
4	14243	1
5	559	1
6	12931	1
7	5071	1
8	355	1
9	2415	1
10	1091	1
11	783	1
12	6947	1
13	8367	1
14	9731	1
15	591	1
16	1251	1
17	2031	1
18	6083	1
19	4495	1
20	16035	1
21	16175	1
22	6531	1
23	12495	1
24	2147	1
25	1647	1
26	11075	1
27	8207	1
28	8739	1
29	7599	1
30	3331	1
31	8015	1
32	3043	1
33	1263	1
34	16067	1
35	11919	1
36	1443	1
37	15407	1
38	131	1
39	3535	1
40	3939	1
41	879	1

Iteraciones	Números generados	Repeticiones
42	4675	1
43	15631	1
44	10531	1
45	6831	1
46	13315	1
47	15439	1
48	4835	1
49	495	1
50	9667	1
51	2959	1
52	3235	1
53	14639	1
54	10115	1
55	10959	1
56	5731	1
57	111	1
58	14659	1
59	6671	1
60	12323	1
61	6063	1
62	6915	1
63	6479	1
64	6627	1
65	16111	1
66	3267	1
67	10383	1
68	5027	1
69	13871	1
70	3715	1
71	1999	1
72	7523	1
73	15727	1
74	8259	1
75	14095	1
76	14115	1
77	5295	1
78	515	1
79	13903	1
80	8419	1
81	15343	1
82	13251	1
83	1423	1
84	6819	1
85	13103	1
86	13699	1
87	9423	1
88	9315	1
89	14959	1

Iteraciones	Números generados	Repeticiones
90	1859	1
91	5135	1
92	15907	1
93	4527	1
94	10499	1
95	4943	1
96	10211	1
97	14575	1
98	6851	1
99	8847	1
100	8611	1

Intervalo	Desde	Hasta	Observados
1	0	1638	14
2	1639	3276	11
3	3277	4915	8
4	4916	6553	10
5	6554	8191	10
6	8192	9830	11
7	9831	11468	7
8	11469	13106	6
9	13107	14744	12
10	14745	16383	11

Cuadro 2: Prueba de uniformidad – 100 números

Intervalo	Desde	Hasta	Observados
1	0	1638	6
2	1639	3276	3
3	3277	4915	1
4	4916	6553	2
5	6554	8191	1
6	8192	9830	2
7	9831	11468	0
8	11469	13106	2
9	13107	14744	2
10	14745	16383	1

Cuadro 3: Prueba de uniformidad – primeros 20 números

6. Conclusiones

- Se logró implementar de forma exitosa un generador de números pseudoaleatorios en Java utilizando un método congruencial de tipo cuadrático, demostrando que es posible producir secuencias numéricas con apariencia aleatoria a partir de una fórmula puramente determinística.
- El comportamiento de la secuencia generada depende críticamente de la elección de los parámetros (semilla, multiplicador, incremento y módulo). Una mala selección de estos valores puede provocar periodicidad corta o patrones repetitivos en los números generados, lo cual reduce la calidad del generador.
- El uso del operador módulo es fundamental en este tipo de algoritmos, ya que permite acotar el rango de los resultados y evitar el desbordamiento de los tipos de datos numéricos, aunque en implementaciones con valores muy grandes debe tenerse especial cuidado con la capacidad del tipo `long`.
- A diferencia de un generador congruencial lineal clásico, la incorporación del término cuadrático (X_n^2) añade una componente no lineal que puede mejorar la dispersión de los valores, pero también puede generar comportamientos menos predecibles o ciclos irregulares que ameritan un análisis estadístico más profundo (por ejemplo, pruebas de uniformidad o de independencia).
- Este tipo de ejercicios permite comprender que los "números aleatorios" generados por un computador no son verdaderamente aleatorios, sino pseudoaleatorios, ya que siempre parten de una semilla inicial y siguen una regla determinística; por ello, ante la misma semilla, el programa siempre producirá exactamente la misma secuencia de números.